Time Series Data Wrangling — Practice Exercise Set

Prepared by: Yadu

Focus areas: creating a date spine, lag/lead functions, rolling windows & moving averages, and aggregating to different frequencies. These are the day-to-day pandas skills behind almost every real time series project — before you can decompose or forecast anything, the data usually needs this kind of shaping first.

Datasets

File	Frequency	Rows	Period	Notice
`store_transactions_sparse.csv`	Daily (with gaps)	405	2023-01-01 to 2024-02-29	20 calendar days are missing — the store was closed (holidays + a few random outages) and no row was logged
`energy_usage_hourly.csv`	Hourly (with gaps)	709	2024-03-01 to 2024-03-30	11 hours are missing — two sensor-downtime windows

Columns:

- `store_transactions_sparse.csv` — `date`, `transactions`, `revenue`
- `energy_usage_hourly.csv` — `timestamp`, `kwh`

Both files are **intentionally incomplete** — that's the point. A real calendar day or hour that has no row is the exact problem the "date spine" section below is designed to fix.

Part A — Creating the Date Spine

A **date spine** is a complete, gap-free sequence of dates/timestamps that you generate yourself and then attach your real (possibly sparse) data onto — so every expected period is represented, even the ones with no recorded activity.

1. Load `store_transactions_sparse.csv` with `date` parsed as a datetime. How many rows does it have? How many calendar days actually exist between its minimum and maximum date? What's the difference, and what does that difference represent?
2. Build a complete daily date spine from the minimum to the maximum date using `pd.date_range()`. Reindex (or merge) the sparse data onto this spine so every calendar day has a row.
3. After reindexing, `transactions` and `revenue` will be `NaN` on the days that were missing. Decide how to fill them — should a missing day become `0` (the store made no sales) or should it be interpolated from neighboring days? Justify your choice, then implement it.
4. Add a boolean column `was_closed` that is `True` for every date that had to be filled in by the spine (i.e., wasn't in the original sparse file) and `False` otherwise. This preserves the information you'd otherwise lose by filling with zeros.
5. Repeat the spine-building process for `energy_usage_hourly.csv`, this time building an **hourly** spine. For the two sensor-downtime gaps, would filling with `0` make physical sense the way it did for store closures? Choose and implement a more appropriate fill method (e.g., linear interpolation), and explain why it's a better fit here than for Exercise 3.

Part B — Lag and Lead Functions

Once your data sits on a complete date spine, lag/lead features (values shifted backward or forward in time) unlock comparisons like "vs. yesterday" or "vs. this time last week."

6. On the spined store data from Part A, create a `lag_1` column (yesterday's revenue) and a `lag_7` column (revenue exactly one week earlier, same weekday) using `.shift()`.
7. Create a `lead_1` column (tomorrow's revenue). In a forecasting model that predicts today's revenue, would it ever be legitimate to use a lead feature as a model *input*? Explain the data-leakage risk in one or two sentences.
8. Compute day-over-day change as `revenue - lag_1`, and separately compute day-over-day **percent** change using `.pct_change()`. Confirm both approaches agree (within

rounding) on a handful of rows.

9. With only ~14 months of data, a 365-day lag would leave almost no usable rows. What lag would you use instead to compare "this week's Saturday" to "last week's Saturday," and why is `lag_7` more useful here than `lag_1` for spotting *weekly* seasonality?
10. Using `.shift()`, create a boolean `record_day` column that flags a date as a "record day" when its revenue is higher than **both** `lag_7` and `lag_14` (i.e., beats the same weekday from the previous two weeks).

Part C — Rolling Windows and Moving Averages

Rolling windows summarize a trailing (or centered) slice of recent history — the backbone of smoothing, trend-spotting, and anomaly flags.

11. Compute a **7-day trailing** rolling mean of `revenue` using `.rolling(window=7).mean()`. Why is a trailing window (rather than centered) the right choice if this number will be shown on a live "as of today" dashboard?
12. Now compute a **7-day centered** rolling mean (`center=True`) on the same column. Compare a handful of matching dates between Q11 and Q12 — describe the difference in plain terms, and give one situation where a centered window is the better (or only valid) choice.
13. Compute a 7-day rolling standard deviation of `revenue`. Combine it with the rolling mean from Q11 to build a simple z-score-style flag: mark a day as `unusual = True` when `abs(revenue - rolling_mean) > 2 * rolling_std`. How many days get flagged?
14. The first 6 rows of any 7-day rolling calculation are `NaN` by default. Recompute the Q11 rolling mean using `min_periods=1` instead of the default. What trade-off are you accepting at the start of the series by doing this (think about how reliable a "7-day average" really is when it's only averaging 1–2 real days)?
15. Compute an **expanding** (cumulative, growing-window) mean of `revenue` from the first date onward using `.expanding().mean()`. In one sentence, explain how an expanding window is conceptually different from a rolling window of fixed size.

Part D — Aggregating to Different Frequencies

Once daily (or hourly) data is clean, it's often summarized to a coarser frequency for reporting — weekly, monthly, or quarterly.

16. Resample the spined daily store data to **weekly** totals using `.resample("W").agg(...)`, summing both `transactions` and `revenue`. How many

complete weeks does the resulting table contain?

17. Now resample to **monthly**. If you wanted an "average transaction value" column at the monthly level (revenue per transaction), would you get the right answer by resampling a *daily* average-transaction-value column with `.mean()`? Explain why summing/averaging an already-averaged ratio can be misleading, and show the correct way to compute it (hint: aggregate the raw totals first, then divide).
18. Downsample `energy_usage_hourly.csv` to **daily** totals (sum of `kwh`) and then to **weekly** totals. As a check, take the daily totals and **upsample** them back to hourly using `.resample("h").asfreq()`. What do the new hourly rows contain, and what fill method (`ffill`, `bfill`, or interpolation) would make the upsampled data usable — and why does upsampling raise more judgment calls than downsampling?
19. Using `groupby` with `pd.Grouper` (or simply grouping by `.dt.dayofweek`), compute the **average revenue per weekday** across the whole 14-month store dataset, and present it as a small table (Monday through Sunday).
20. Compute total monthly revenue two different ways: (a) `.resample("MS").sum()` directly on the daily data, and (b) resample to weekly sums first, then re-aggregate those weekly sums up to months. Do the two totals match exactly for every month? Explain a scenario where they would *not* match (hint: think about what happens when a week straddles the boundary between two months).

Part E — Mini Integration Challenge

21. Using `store_transactions_sparse.csv`, write one script that chains everything above into a single clean output table: build the date spine, fill gaps appropriately, compute a 7-day rolling average of daily revenue, resample to a **weekly summary** with total revenue, total transactions, and average daily revenue, then add a **4-week rolling average of weekly revenue** and a **week-over-week percent change** column (using `lag_1` on the weekly table). The result should be a single tidy weekly table ready to hand to a manager.

Tips

- `pd.date_range(start, end, freq="D")` builds the spine; `.set_index("date").reindex(spine)` attaches your real data to it.
- `.shift(n)` looks *backward* n periods (lag); `.shift(-n)` looks *forward* n periods (lead).

- `.rolling(window=k)` is trailing by default; pass `center=True` to center it. `.expanding()` grows from the start instead of sliding.
- `.resample(rule)` needs a `DatetimeIndex` — common rules: `"D"`, `"W"`, `"MS"` (month start), `"QS"` (quarter start), `"h"`.
- Downsampling (`D → W`) needs an aggregation (`.sum()`, `.mean()`); upsampling (`D → h`) needs a fill method (`.ffill()`, `.interpolate()`) since there's no real data for the new, finer time slots.